```
%pip install tensorflow numpy
```

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

```
sentence = "the cat sits on the mat"

words = sentence.split()

vocab = sorted(set(words))

word_to_id = {word: i for i, word in enumerate(vocab)}
id_to_word = {i: word for word, i in word_to_id.items()}

print("Words:", words)
print("Vocabulary:", word_to_id)
```

```
X = []
y = []

for i in range(1, len(words) - 1):
    context = [
        word_to_id[words[i - 1]],
        word_to_id[words[i + 1]]
    ]

    target = word_to_id[words[i]]

    X.append(context)
    y.append(target)

X = np.array(X)
y = np.array(y)

print("Training input (X):")
print(X)

print("Target output (y):")
print(y)
```

```
model = keras.Sequential([
    layers.Input(shape=(2,)),
```

```
    layers.Embedding(len(vocab), 10),
    layers.GlobalAveragePooling1D(),
    layers.Dense(len(vocab), activation="softmax")
])

model.summary()
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
model.fit(X, y, epochs=200, verbose=0)

print("Model training completed!")
```

```
context_words = ["cat", "on"]

context_ids = np.array([[
    word_to_id[context_words[0]],
    word_to_id[context_words[1]]
]])

prediction = model.predict(context_ids, verbose=0)

predicted_id = np.argmax(prediction)
predicted_word = id_to_word[predicted_id]

print("Context words:", context_words)
print("Predicted word:", predicted_word)
print("Probability:", round(float(prediction[0][predicted_id]), 3))
```